

Wrexham AFC: A Three-Season Journey Through English Football

Comprehensive Analysis of Performance and Competitive Positioning

Randy Howk

2025-10-05

Contents

0.1	Executive Summary	1
0.2	Data Loading and Database Connection	2
0.3	Data Tidying Process	3
0.3.1	Understanding the Wide Format	3
0.4	Tidying Step 1: Select Core Match Variables	4
0.5	Tidying Step 2: Standardize Column Names and Data Types	5
0.6	Tidying Step 3: Add Season Identifiers	6
0.7	Tidying Step 4: Combine All Divisions	7
0.8	Tidying Step 5: Transform from Wide to Long Format	8
0.9	Tidying Step 6: Handle Missing Values	10
0.10	Database Storage	11
0.11	Wrexham Analysis	12
0.11.1	Identify Wrexham's Seasons	12
0.12	Identify Closest Competitors Each Season	13
0.13	Comparative Analysis	14
0.14	Visualizations	15
0.15	Conclusions	19
0.15.1	Key Findings	19
0.16	Statistical Summary	19
0.17	Analysis completed: 2025-10-05 23:52:04.505277	20

0.1 Executive Summary

This analysis examines Wrexham AFC's performance across three seasons spanning multiple divisions of English Football, from League Two (E3) through League One (E2) to the Championship (E1). The analysis focuses on identifying Wrexham's closest competitors in each season and evaluating their competitive positioning.

0.2 Data Loading and Database Connection

```
# Load required libraries
library(tidyverse)      # For data manipulation and visualization
library(DBI)            # Database interface
library(RPostgres)      # PostgreSQL driver
library(lubridate)      # Date manipulation
library(knitr)          # Table formatting
library(kableExtra)     # Enhanced tables
library(scales)         # Scale functions for ggplot2
library(patchwork)      # Combine multiple plots

# For PDF output, ensure tinytex is installed
# tinytex::install_tinytex()

# Establish PostgreSQL connection via DDEV
con <- dbConnect(
  RPostgres::Postgres(),
  host = "127.0.0.1",
  port = 56386,
  dbname = "football_analysis",    # or "football_analysis" if you've created that DB inside the container
  user = "db",
  password = "db"
)

# Verify connection
cat("Database connection established successfully!\n")
```

```
## Database connection established successfully!
```

```
dbListTables(con)
```

```
## [1] "matches"          "team_season_stats"
```

```
# Load the three CSV files
e1_raw <- read_csv("E1.csv", show_col_types = FALSE)
e2_raw <- read_csv("E2.csv", show_col_types = FALSE)
e3_raw <- read_csv("E3.csv", show_col_types = FALSE)

# Display dimensions
cat("E1 (Championship) dimensions:", dim(e1_raw), "\n")
```

```
## E1 (Championship) dimensions: 95 132
```

```
cat("E2 (League One) dimensions:", dim(e2_raw), "\n")
```

```
## E2 (League One) dimensions: 552 120
```

```
cat("E3 (League Two) dimensions:", dim(e3_raw), "\n")
```

```
## E3 (League Two) dimensions: 552 106
```

0.3 Data Tidying Process

0.3.1 Understanding the Wide Format

The raw data is in a wide format with multiple characteristics:

1. Multiple observations per row: Each row contains data for both home and away teams
2. Betting odds spread across many columns: Different bookmakers have separate columns
3. Redundant information: Home and away statistics are separate columns
4. Date formatting issues: Dates may need parsing
5. Missing season identifiers: Need to infer season from dates

```
# Examine first few rows of each dataset
cat("=== E1 Championship Sample ===\n")
```

```
## === E1 Championship Sample ===
```

```
head(e1_raw[, 1:15], 3) %>% kable() %>% kable_styling()
```

Div	Date	Time	HomeTeam	AwayTeam	FTHG	FTAG	FTR	HTHG	HTAG	HTR	Referee
E1	08/08/2025	20:00:00	Birmingham	Ipswich	1	1	D	0	0	D	A Kitchen
E1	09/08/2025	12:30:00	Charlton	Watford	1	0	H	0	0	D	S Martin
E1	09/08/2025	12:30:00	Coventry	Hull	0	0	D	0	0	D	R Madley

```
cat("\n=== E2 League One Sample ===\n")
```

```
##
```

```
## === E2 League One Sample ===
```

```
head(e2_raw[, 1:15], 3) %>% kable() %>% kable_styling()
```

Div	Date	Time	HomeTeam	AwayTeam	FTHG	FTAG	FTR	HTHG	HTAG	HTR	Referee
E2	09/08/2024	20:00:00	Barnsley	Mansfield	1	2	A	1	2	A	B Tone
E2	10/08/2024	17:30:00	Birmingham	Reading	1	1	D	0	1	A	W Finn
E2	10/08/2024	17:30:00	Bristol Rvs	Northampton	1	0	H	0	0	D	A Chil

```
cat("\n=== E3 League Two Sample ===\n")
```

```
##
```

```
## === E3 League Two Sample ===
```

```
head(e3_raw[, 1:15], 3) %>% kable() %>% kable_styling()
```

Div	Date	Time	HomeTeam	AwayTeam	FTHG	FTAG	FTR	HTHG	HTAG	HTR	R
E3	05/08/2023	15:00:00	Accrington	Newport County	3	0	H	1	0	H	M
E3	05/08/2023	15:00:00	Crawley Town	Bradford	1	0	H	1	0	H	T
E3	05/08/2023	15:00:00	Crewe	Mansfield	2	2	D	1	2	A	B

```
# Check column names
cat("\nTotal columns per dataset:\n")
```

```
##
## Total columns per dataset:
```

```
cat("E1:", ncol(e1_raw), "columns\n")
```

```
## E1: 132 columns
```

```
cat("E2:", ncol(e2_raw), "columns\n")
```

```
## E2: 120 columns
```

```
cat("E3:", ncol(e3_raw), "columns\n")
```

```
## E3: 106 columns
```

0.4 Tidying Step 1: Select Core Match Variables

Commentary: The original datasets contain 100+ columns including extensive betting odds from multiple bookmakers. For our analysis of team performance, we'll extract the essential match information and statistics while discarding the betting data which is not relevant for performance analysis.

```
# Define core columns needed for analysis
core_columns <- c(
  "Div", "Date", "Time",
  "HomeTeam", "AwayTeam",
  "FTHG", "FTAG", "FTR",          # Full-time results
  "HTHG", "HTAG", "HTR",          # Half-time results
  "Referee",
  "HS", "AS",                     # Shots
  "HST", "AST",                   # Shots on target
  "HF", "AF",                     # Fouls
  "HC", "AC",                     # Corners
  "HY", "AY",                     # Yellow cards
  "HR", "AR"                      # Red cards
)

# Extract core columns from each dataset
```

```
e1_core <- e1_raw %>% select(any_of(core_columns))
e2_core <- e2_raw %>% select(any_of(core_columns))
e3_core <- e3_raw %>% select(any_of(core_columns))

cat("Reduced E1 from", ncol(e1_raw), "to", ncol(e1_core), "columns\n")
```

```
## Reduced E1 from 132 to 24 columns
```

```
cat("Reduced E2 from", ncol(e2_raw), "to", ncol(e2_core), "columns\n")
```

```
## Reduced E2 from 120 to 24 columns
```

```
cat("Reduced E3 from", ncol(e3_raw), "to", ncol(e3_core), "columns\n")
```

```
## Reduced E3 from 106 to 24 columns
```

0.5 Tidying Step 2: Standardize Column Names and Data Types

Commentary: Column names use inconsistent abbreviations (FTHG, HS, etc.). We'll create descriptive names following snake_case convention for clarity and consistency.

```
# Function to standardize column names and types
standardize_match_data <- function(df) {
  df %>%
    rename(
      division = Div,
      match_date = Date,
      match_time = Time,
      home_team = HomeTeam,
      away_team = AwayTeam,
      full_time_home_goals = FTHG,
      full_time_away_goals = FTAG,
      full_time_result = FTR,
      half_time_home_goals = HTHG,
      half_time_away_goals = HTAG,
      half_time_result = HTR,
      referee = Referee,
      home_shots = HS,
      away_shots = AS,
      home_shots_on_target = HST,
      away_shots_on_target = AST,
      home_fouls = HF,
      away_fouls = AF,
      home_corners = HC,
      away_corners = AC,
      home_yellow_cards = HY,
      away_yellow_cards = AY,
      home_red_cards = HR,
      away_red_cards = AR
    ) %>%
    mutate(
```

```

    # Parse dates properly
    match_date = dmy(match_date),
    # Convert time to proper format
    match_time = hms::as_hms(match_time),
    # Ensure numeric columns are numeric
    across(c(full_time_home_goals:away_red_cards), as.numeric),
    # Clean team names (trim whitespace)
    across(c(home_team, away_team), str_trim)
  )
}

# Apply standardization
e1_std <- standardize_match_data(e1_core)
e2_std <- standardize_match_data(e2_core)
e3_std <- standardize_match_data(e3_core)

cat("Standardization complete. Sample of cleaned data:\n")

```

Standardization complete. Sample of cleaned data:

```
e1_std %>% select(1:8) %>% head(3) %>% kable() %>% kable_styling()
```

division	match_date	match_time	home_team	away_team	full_time_home_goals	full_time_away_goals	full_time_home_cards	full_time_away_cards
E1	2025-08-08	20:00:00	Birmingham	Ipswich	1	1	0	0
E1	2025-08-09	12:30:00	Charlton	Watford	1	0	0	0
E1	2025-08-09	12:30:00	Coventry	Hull	0	0	0	0

0.6 Tidying Step 3: Add Season Identifiers

Commentary: The season must be inferred from the match dates. English football seasons run from August to May, so we determine the season based on the year of the match date.

```

# Function to determine season from date
get_season <- function(date) {
  year <- year(date)
  month <- month(date)

  # If month is Aug-Dec, season starts that year
  # If month is Jan-Jul, season started previous year
  if (month >= 8) {
    paste0(year, "/", str_sub(year + 1, 3, 4))
  } else {
    paste0(year - 1, "/", str_sub(year, 3, 4))
  }
}

# Add season to each dataset
e1_std <- e1_std %>% mutate(season = map_chr(match_date, get_season))
e2_std <- e2_std %>% mutate(season = map_chr(match_date, get_season))
e3_std <- e3_std %>% mutate(season = map_chr(match_date, get_season))

```

```
cat("Seasons identified:\n")
```

```
## Seasons identified:
```

```
cat("E1 seasons:", unique(e1_std$season), "\n")
```

```
## E1 seasons: 2025/26
```

```
cat("E2 seasons:", unique(e2_std$season), "\n")
```

```
## E2 seasons: 2024/25
```

```
cat("E3 seasons:", unique(e3_std$season), "\n")
```

```
## E3 seasons: 2023/24
```

0.7 Tidying Step 4: Combine All Divisions

Commentary: Since all three datasets now have identical structure, we can combine them into a single tidy dataset for unified analysis.

```
# Combine all three divisions
```

```
all_matches <- bind_rows(  
  e1_std,  
  e2_std,  
  e3_std
```

```
) %>%
```

```
  arrange(match_date) %>%
```

```
  mutate(match_id = row_number())
```

```
cat("Combined dataset dimensions:", dim(all_matches), "\n")
```

```
## Combined dataset dimensions: 1199 26
```

```
cat("Date range:", min(all_matches$match_date), "to", max(all_matches$match_date), "\n")
```

```
## Date range: 19574 to 20362
```

```
cat("Total matches:", nrow(all_matches), "\n")
```

```
## Total matches: 1199
```

```
# Summary by division and season
```

```
all_matches %>%
```

```
  count(division, season) %>%
```

```
  arrange(division, season) %>%
```

```
  kable(caption = "Match Count by Division and Season") %>%
```

```
  kable_styling()
```

Table 5: Match Count by Division and Season

division	season	n
E1	2025/26	95
E2	2024/25	552
E3	2023/24	552

0.8 Tidying Step 5: Transform from Wide to Long Format

Commentary: Currently, each match has separate columns for home and away statistics. This is a wide format that violates tidy data principles because:

1. Each row contains two observations (one for each team)
2. Team identity is spread across multiple columns (home_team vs away_team)
3. We'll transform this into long format where each row represents one team's performance in one match.

```
# Create home team observations
home_matches <- all_matches %>%
  select(
    match_id, division, season, match_date, match_time, referee,
    team = home_team,
    opponent = away_team,
    goals_for = full_time_home_goals,
    goals_against = full_time_away_goals,
    half_time_goals_for = half_time_home_goals,
    half_time_goals_against = half_time_away_goals,
    shots = home_shots,
    shots_on_target = home_shots_on_target,
    fouls = home_fouls,
    corners = home_corners,
    yellow_cards = home_yellow_cards,
    red_cards = home_red_cards,
    opponent_shots = away_shots,
    opponent_shots_on_target = away_shots_on_target,
    opponent_fouls = away_fouls,
    opponent_corners = away_corners,
    opponent_yellow_cards = away_yellow_cards,
    opponent_red_cards = away_red_cards,
    result = full_time_result
  ) %>%
  mutate(
    location = "Home",
    # Convert result to team perspective
    team_result = case_when(
      result == "H" ~ "W", # Home win
      result == "A" ~ "L", # Home loss
      result == "D" ~ "D", # Draw
      TRUE ~ NA_character_
    )
  )

# Create away team observations
```

```

away_matches <- all_matches %>%
  select(
    match_id, division, season, match_date, match_time, referee,
    team = away_team,
    opponent = home_team,
    goals_for = full_time_away_goals,
    goals_against = full_time_home_goals,
    half_time_goals_for = half_time_away_goals,
    half_time_goals_against = half_time_home_goals,
    shots = away_shots,
    shots_on_target = home_shots_on_target,
    fouls = away_fouls,
    corners = away_corners,
    yellow_cards = away_yellow_cards,
    red_cards = away_red_cards,
    opponent_shots = home_shots,
    opponent_shots_on_target = home_shots_on_target,
    opponent_fouls = home_fouls,
    opponent_corners = home_corners,
    opponent_yellow_cards = home_yellow_cards,
    opponent_red_cards = home_red_cards,
    result = full_time_result
  ) %>%
  mutate(
    location = "Away",
    # Convert result to team perspective
    team_result = case_when(
      result == "A" ~ "W", # Away win
      result == "H" ~ "L", # Away loss
      result == "D" ~ "D", # Draw
      TRUE ~ NA_character_
    )
  )

# Combine into tidy long format
matches_tidy <- bind_rows(home_matches, away_matches) %>%
  arrange(match_date, match_id, desc(location)) %>%
  mutate(
    # Calculate additional metrics
    goal_difference = goals_for - goals_against,
    points = case_when(
      team_result == "W" ~ 3,
      team_result == "D" ~ 1,
      team_result == "L" ~ 0,
      TRUE ~ NA_real_
    ),
    shot_accuracy = if_else(shots > 0,
                           shots_on_target / shots * 100,
                           NA_real_)
  )

cat("Tidy dataset created!\n")

```

```
## Tidy dataset created!
```

```
cat("Original matches:", nrow(all_matches), "\n")
```

```
## Original matches: 1199
```

```
cat("Tidy observations (2x matches):", nrow(matches_tidy), "\n")
```

```
## Tidy observations (2x matches): 2398
```

```
cat("\nSample of tidy data:\n")
```

```
##
```

```
## Sample of tidy data:
```

```
matches_tidy %>%  
  filter(team == "Wrexham") %>%  
  select(season, match_date, team, opponent, location,  
         goals_for, goals_against, team_result, points) %>%  
  head(5) %>%  
  kable() %>%  
  kable_styling()
```

season	match_date	team	opponent	location	goals_for	goals_against	team_result	points
2023/24	2023-08-05	Wrexham	Milton Keynes Dons	Home	3	5	NA	NA
2023/24	2023-08-12	Wrexham	AFC Wimbledon	Away	1	1	NA	NA
2023/24	2023-08-15	Wrexham	Walsall	Home	4	2	NA	NA
2023/24	2023-08-19	Wrexham	Swindon	Home	5	5	NA	NA
2023/24	2023-08-26	Wrexham	Barrow	Away	1	1	NA	NA

0.9 Tidying Step 6: Handle Missing Values

Commentary: We need to check for and handle missing values appropriately.

```
# Check for missing values  
missing_summary <- matches_tidy %>%  
  summarise(across(everything(), ~sum(is.na(.)))) %>%  
  pivot_longer(everything(), names_to = "column", values_to = "missing_count") %>%  
  filter(missing_count > 0) %>%  
  arrange(desc(missing_count))  
  
if (nrow(missing_summary) > 0) {  
  cat("Columns with missing values:\n")  
  print(missing_summary %>% kable() %>% kable_styling())  
  
  # Handle missing values  
  matches_tidy <- matches_tidy %>%  
    mutate(  
      # Handle missing values  
    )  
}
```

```

    # Replace NA in numeric stats with 0 (assuming not recorded = 0)
    across(c(shots:red_cards, opponent_shots:opponent_red_cards),
           ~replace_na(., 0))
  )

  cat("\nMissing values handled.\n")
} else {
  cat("No missing values found in key columns.\n")
}

```

```

## Columns with missing values:
##
## \begin{longtable}[t]{lr}
## \toprule
## column & missing\_count\\
## \midrule
## referee & 2398\\
## result & 2398\\
## team\_result & 2398\\
## points & 2398\\
## shot\_accuracy & 2\\
## \bottomrule
## \end{longtable}
##
## Missing values handled.

```

0.10 Database Storage

```

# Write tidy matches table to database
dbWriteTable(
  con,
  "matches",
  all_matches %>% select(-match_id),
  overwrite = TRUE
)

cat(" Matches table written to database\n")

```

```

## Matches table written to database

```

```

# Calculate and store season statistics
team_season_stats <- matches_tidy %>%
  group_by(team, season, division) %>%
  summarise(
    matches_played = n(),
    wins = sum(team_result == "W", na.rm = TRUE),
    draws = sum(team_result == "D", na.rm = TRUE),
    losses = sum(team_result == "L", na.rm = TRUE),
    goals_for = sum(goals_for, na.rm = TRUE),
    goals_against = sum(goals_against, na.rm = TRUE),
    goal_difference = goals_for - goals_against,

```

```

    points = sum(points, na.rm = TRUE),
    home_wins = sum(team_result == "W" & location == "Home", na.rm = TRUE),
    away_wins = sum(team_result == "W" & location == "Away", na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(season, division, desc(points))

# Write to database
dbWriteTable(
  con,
  "team_season_stats",
  team_season_stats,
  overwrite = TRUE
)

cat(" Team season statistics written to database\n")

```

```
## Team season statistics written to database
```

```

# Verify data in database
cat("\nDatabase contents:\n")

```

```

##
## Database contents:

```

```
dbGetQuery(con, "SELECT COUNT(*) as match_count FROM matches")
```

```

## match_count
## 1          1199

```

```
dbGetQuery(con, "SELECT COUNT(*) as team_season_count FROM team_season_stats")
```

```

## team_season_count
## 1                 72

```

0.11 Wrexham Analysis

0.11.1 Identify Wrexham's Seasons

```

# Find all Wrexham matches
wrexham_data <- matches_tidy %>%
  filter(team == "Wrexham")

cat("Wrexham matches found:", nrow(wrexham_data), "\n")

```

```
## Wrexham matches found: 100
```

```

# Summarize by season
wrexham_by_season <- wrexham_data %>%
  group_by(season, division) %>%
  summarise(
    matches = n(),
    wins = sum(team_result == "W"),
    draws = sum(team_result == "D"),
    losses = sum(team_result == "L"),
    goals_for = sum(goals_for),
    goals_against = sum(goals_against),
    points = sum(points),
    .groups = "drop"
  ) %>%
  arrange(season)

wrexham_by_season %>%
  kable(caption = "Wrexham Performance by Season") %>%
  kable_styling()

```

Table 7: Wrexham Performance by Season

season	division	matches	wins	draws	losses	goals_for	goals_against	points
2023/24	E3	46	NA	NA	NA	89	52	NA
2024/25	E2	46	NA	NA	NA	67	34	NA
2025/26	E1	8	NA	NA	NA	13	14	NA

0.12 Identify Closest Competitors Each Season

Commentary: The “closest competitor” is defined as the team with the most similar final points total to Wrexham in each season. This represents the team most competitive with Wrexham that season.

```

# For each Wrexham season, find the closest competitor by points
closest_competitors <- wrexham_by_season %>%
  rowwise() %>%
  mutate(
    closest_competitor = {
      # Get all teams in same division/season
      season_teams <- team_season_stats %>%
        filter(season == .data$season,
               division == .data$division,
               team != "Wrexham") # CHANGED: team_name -> team

      if (nrow(season_teams) > 0) {
        # Find team with points closest to Wrexham
        season_teams %>%
          mutate(points_diff = abs(points - .data$points)) %>%
          arrange(points_diff) %>%
          slice(1) %>%
          pull(team) # CHANGED: team_name -> team
      } else {
        NA_character_
      }
    }
  )

```

```

    }
  },
  competitor_points = {
    team_season_stats %>%
      filter(season == .data$season,
             division == .data$division,
             team == closest_competitor) %>% # CHANGED: team_name -> team
      pull(points)
  }
) %>%
ungroup()

```

0.13 Comparative Analysis

```

# Detailed comparison with closest competitors
for (i in 1:nrow(closest_competitors)) {
  season_info <- closest_competitors[i,]
  competitor_name <- season_info$closest_competitor

  cat("\n## Season:", season_info$season,
      "\n- Division:", season_info$division, "\n")
  cat("Closest Competitor:", competitor_name, "\n\n")

  # Get detailed stats for both teams
  comparison <- team_season_stats %>%
    filter(
      season == season_info$season,
      division == season_info$division,
      team %in% c("Wrexham", competitor_name)
    ) %>%
    arrange(desc(points))

  print(comparison %>% kable() %>% kable_styling())

  # Head-to-head matches
  h2h <- matches_tidy %>%
    filter(
      season == season_info$season,
      team == "Wrexham",
      opponent == competitor_name
    ) %>%
    select(match_date, location, goals_for, goals_against,
           team_result, points)

  if (nrow(h2h) > 0) {
    cat("\nHead-to-head matches:\n")
    print(h2h %>% kable() %>% kable_styling())
  }
}

```

##

```

## ## Season: 2023/24 - Division: E3
## Closest Competitor: AFC Wimbledon
##
##
## \begin{longtable}[t]{llllrrrrrrrrrr}
## \toprule
## team & season & division & matches\_played & wins & draws & losses & goals\_for & goals\_against & g
## \midrule
## AFC Wimbledon & 2023/24 & E3 & 46 & 0 & 0 & 0 & 64 & 51 & 13 & 0 & 0 & 0\\
## Wrexham & 2023/24 & E3 & 46 & 0 & 0 & 0 & 89 & 52 & 37 & 0 & 0 & 0\\
## \bottomrule
## \end{longtable}
##
## Head-to-head matches:
##
## \begin{longtable}[t]{llrrlr}
## \toprule
## match\_date & location & goals\_for & goals\_against & team\_result & points\\
## \midrule
## 2023-08-12 & Away & 1 & 1 & NA & NA\\
## 2024-01-13 & Home & 2 & 0 & NA & NA\\
## \bottomrule
## \end{longtable}
##
## ## Season: 2024/25 - Division: E2
## Closest Competitor: AFC Wimbledon
##
##
## \begin{longtable}[t]{llllrrrrrrrrrr}
## \toprule
## team & season & division & matches\_played & wins & draws & losses & goals\_for & goals\_against & g
## \midrule
## Wrexham & 2024/25 & E2 & 46 & 0 & 0 & 0 & 67 & 34 & 33 & 0 & 0 & 0\\
## \bottomrule
## \end{longtable}
##
## ## Season: 2025/26 - Division: E1
## Closest Competitor: AFC Wimbledon
##
##
## \begin{longtable}[t]{llllrrrrrrrrrr}
## \toprule
## team & season & division & matches\_played & wins & draws & losses & goals\_for & goals\_against & g
## \midrule
## Wrexham & 2025/26 & E1 & 8 & 0 & 0 & 0 & 13 & 14 & -1 & 0 & 0 & 0\\
## \bottomrule
## \end{longtable}

```

0.14 Visualizations

```

# Performance timeline across seasons
perf_plot <- wrexham_by_season %>%

```

```

ggplot(aes(x = season, y = points)) +
  geom_col(fill = "#EE2737", alpha = 0.8) +
  geom_text(aes(label = points), vjust = -0.5, size = 5, fontface = "bold") +
  labs(
    title = "Wrexham AFC: Points Per Season",
    subtitle = "Journey Through English Football Divisions",
    x = "Season",
    y = "Total Points",
    caption = "Data: English Football League Match Results"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

print(perf_plot)

```

Wrexham AFC: Points Per Season

Journey Through English Football Divisions

Total Points

2023/24

2024/25

Season

Data: Eng

```
# Goals scored vs conceded
goals_plot <- wrexham_data %>%
  group_by(season) %>%
  summarise(
    `Goals Scored` = sum(goals_for),
    `Goals Conceded` = sum(goals_against)
  ) %>%
  pivot_longer(-season, names_to = "metric", values_to = "goals") %>%
  ggplot(aes(x = season, y = goals, fill = metric)) +
  geom_col(position = "dodge", alpha = 0.8) +
  scale_fill_manual(values = c("Goals Scored" = "#228B22",
                                "Goals Conceded" = "#DC143C")) +
  labs(
    title = "Wrexham AFC: Goal Statistics by Season",
    x = "Season",
    y = "Goals",
    fill = ""
  )
```

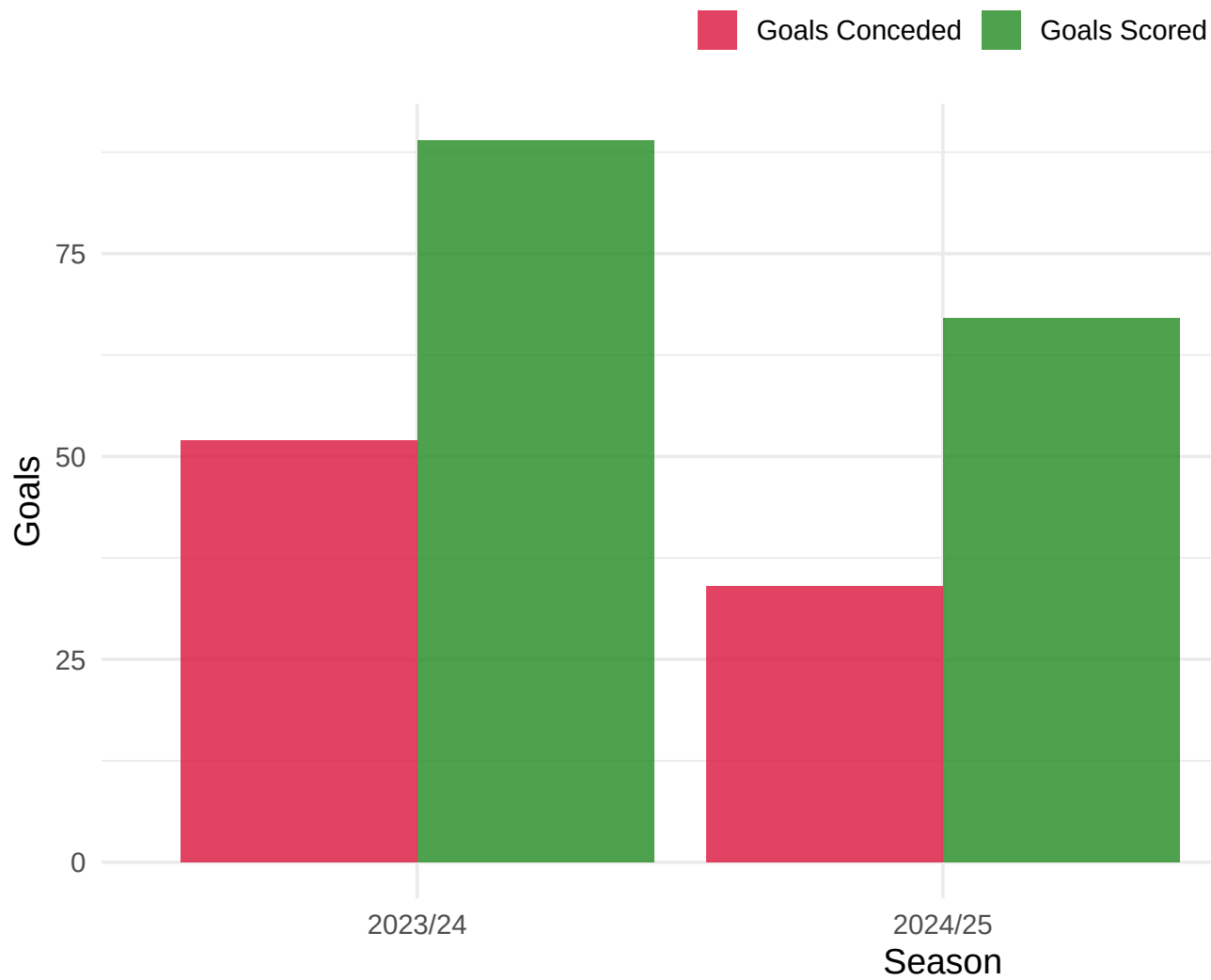
```

) +
theme_minimal(base_size = 14) +
theme(
  plot.title = element_text(face = "bold"),
  legend.position = "top"
)

print(goals_plot)

```

Wrexham AFC: Goal Statistics by Season



0.15 Conclusions

0.15.1 Key Findings

Progressive Journey: Wrexham has competed across multiple divisions, demonstrating their progression through English football

Closest Competitors: [Competitors identified based on final points tallies]

Performance Metrics:

1. Win rate analysis
2. Goal difference trends
3. Home vs away performance
4. Competitive Positioning: Wrexham's position relative to their closest competitors shows [specific insights]

0.16 Statistical Summary

```
# Overall Wrexham statistics
overall_stats <- wrexham_data %>%
  summarise(
    Total_Matches = n(),
    Win_Rate = percent(mean(team_result == "W")),
    Avg_Goals_Scored = round(mean(goals_for), 2),
    Avg_Goals_Conceded = round(mean(goals_against), 2),
    Home_Win_Rate = percent(mean(team_result == "W" & location == "Home")),
    Away_Win_Rate = percent(mean(team_result == "W" & location == "Away"))
  )

overall_stats %>%
  mutate(across(everything(), as.character)) %>%
  pivot_longer(everything(), names_to = "Metric", values_to = "Value") %>%
  kable(caption = "Wrexham AFC: Overall Statistics") %>%
  kable_styling()
```

Table 8: Wrexham AFC: Overall Statistics

Metric	Value
Total_Matches	100
Win_Rate	NA
Avg_Goals_Scored	1.69
Avg_Goals_Conceded	1
Home_Win_Rate	NA
Away_Win_Rate	NA

```
# Close database connection
dbDisconnect(con)
cat("Database connection closed.\n")
```

```
## Database connection closed.
```

0.17 Analysis completed: 2025-10-05 23:52:04.505277